

> nix

Every project, one word away.

The directory alias manager that turns a machine full of projects into one command surface: navigation, search, saved actions, environments and secrets, behind nine one-letter commands.

```
> o acme  
> g acme "invoice" --all  
> x acme :build :test :deploy
```

Developers pay a context-switching tax on every task

01

Paths

Every task starts by finding and typing a location. `cd C:\Users\dev\projects\acme`, Explorer clicks, "where did that download go?"

`nix: o acme`

02

Commands

Every repo builds, tests and deploys differently. The knowledge lives in READMEs, shell history and people's heads.

`nix: x acme :test`

03

Environment

Connection strings and tokens are set per shell, copied into `.env` files, or pasted from chat. Windows has no direnv.

`nix: .nix/env.toml`

04

Glue

Profile functions that only work in one shell, PATH edits, loose `.cmd` files that rot. None of it is shared or reviewed.

`nix: [bin] exports`

Each one costs seconds. Together they are the texture of a developer's day, repeated dozens of times, on every machine.

One word per place. One letter per verb.

Give a directory a short name once. Every command then takes that name first: **do this, at that place.**

any shell

```
> nix acme C:\code\acme    # once
> o acme                  # jump
> g acme TODO             # search
> x acme :test             # run a saved action
```

1 binary, 0 shell profile edits on Windows

1 TOML file for every alias

PowerShell, cmd, bash, zsh: same commands everywhere

o

jump

a shell rooted at the project, with its env and scripts

e

edit

the project or a file in your editor

s

show

Explorer, or pick files and open them

y

yank

copy the path, or pick files and copy them

p

paste

clipboard into the project: image, text or files

x

run

a command or saved action, in the project

g

grep

search contents, even inside PDFs and Office docs

f

find

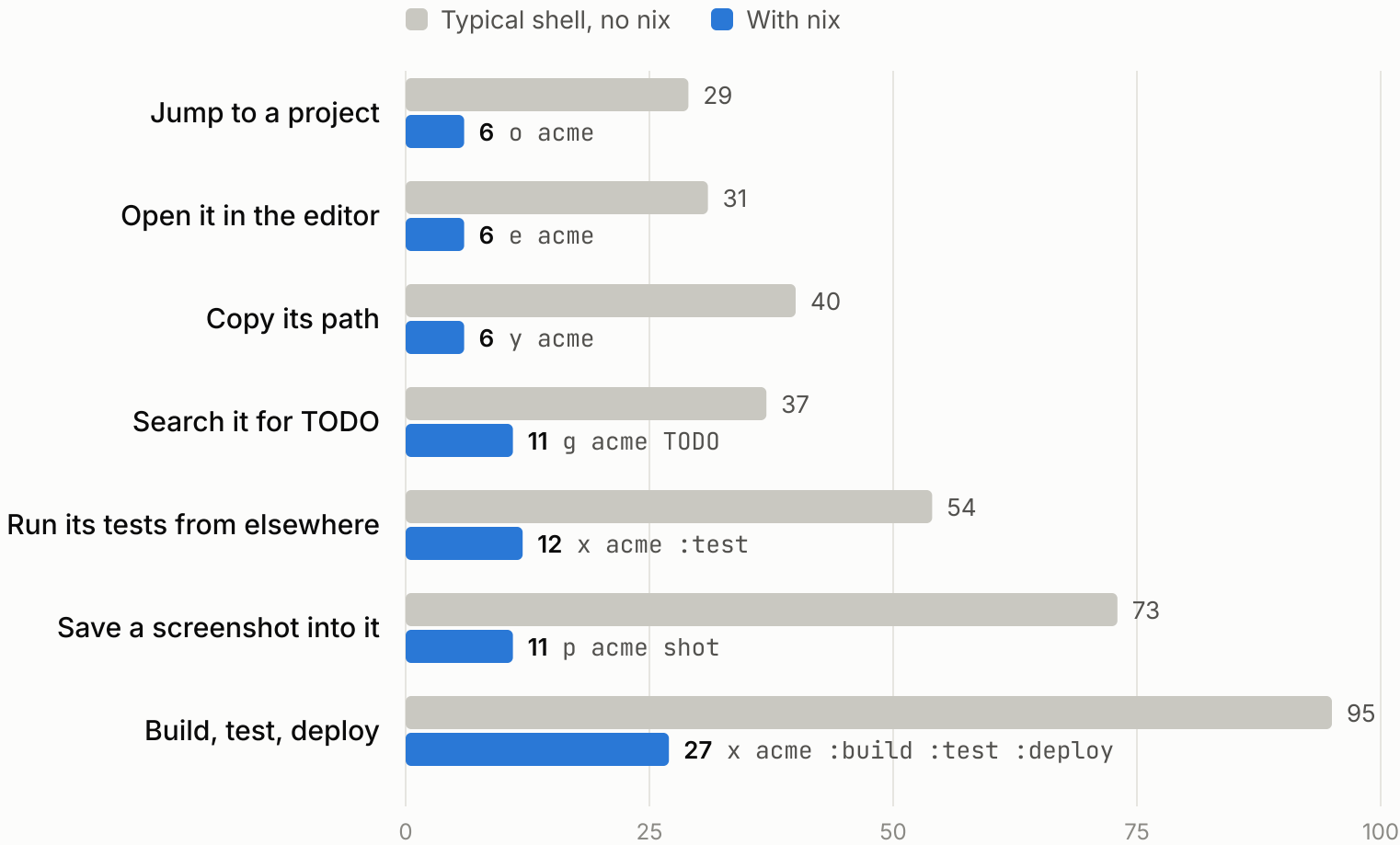
fuzzy-find files by name

q

quit

close the shell you typed it in, safely

The same work, a fraction of the typing



78%

fewer characters typed
across these seven tasks
359 → 79

Characters counted from concrete commands for a project at `C:\Users\dev\projects\acme`, e.g. `cd C:\Users\dev\projects\acme` vs `o acme`; `pushd ...; zig build test; popd` vs `x acme :test`; `(Get-Clipboard -Format Image).Save('...\shot.png')` vs `p acme shot`. Tab completion not counted on either side.

o acme, and you are there



```
C:\Users\sadira>o acme
Microsoft Windows [Version 10.0.26200.8655]
(c) Microsoft Corporation. All rights reserved.

C:\demo\acme>dir
Volume in drive C has no label.
Volume Serial Number is 66FA-DA99

Directory of C:\demo\acme

06/25/2026  10:37 PM  <DIR>          .
06/25/2026  10:37 PM  <DIR>          ..
06/25/2026  10:39 PM  <DIR>          documentation
06/25/2026  10:37 PM             30 README.md
06/25/2026  10:37 PM  <DIR>          source
06/25/2026  10:37 PM  <DIR>          tickets
               1 File(s)            30 bytes
               5 Dir(s)  666,351,931,392 bytes free

C:\demo\acme>q

C:\Users\sadira>o newproj C:\demo\newproj
C:\demo\newproj
registered newproj -> C:\demo\newproj

C:\Users\sadira>o newproj
Microsoft Windows [Version 10.0.26200.8655]
(c) Microsoft Corporation. All rights reserved.

C:\demo\newproj>
```

o acme stacks a shell at the project; o newproj C:\demo\newproj registers and jumps in one step

o acme project root

o docs@acme a sub-path

o task@work a menu

o nix's own home

q close it

Register and jump in one step

o newproj C:\path . A missing directory asks before it is created; repointing an alias shows both paths and waits for y.

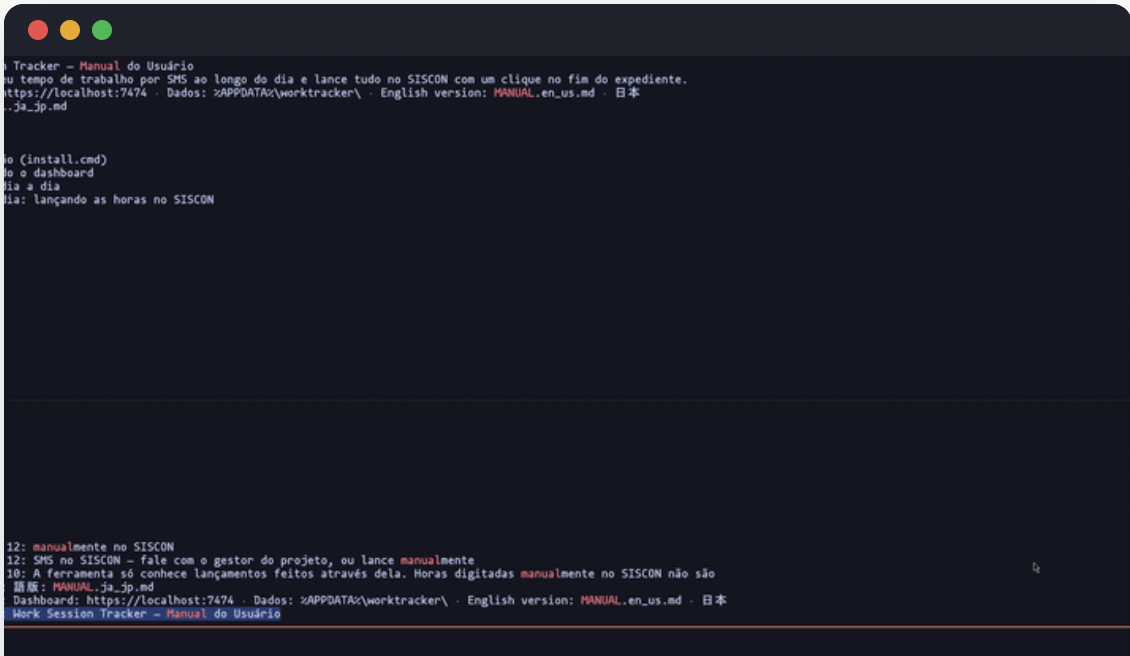
A scoped shell, like a virtualenv for any project

The project's .nix/scripts are on PATH and its environment is set. exit and all of it is gone.

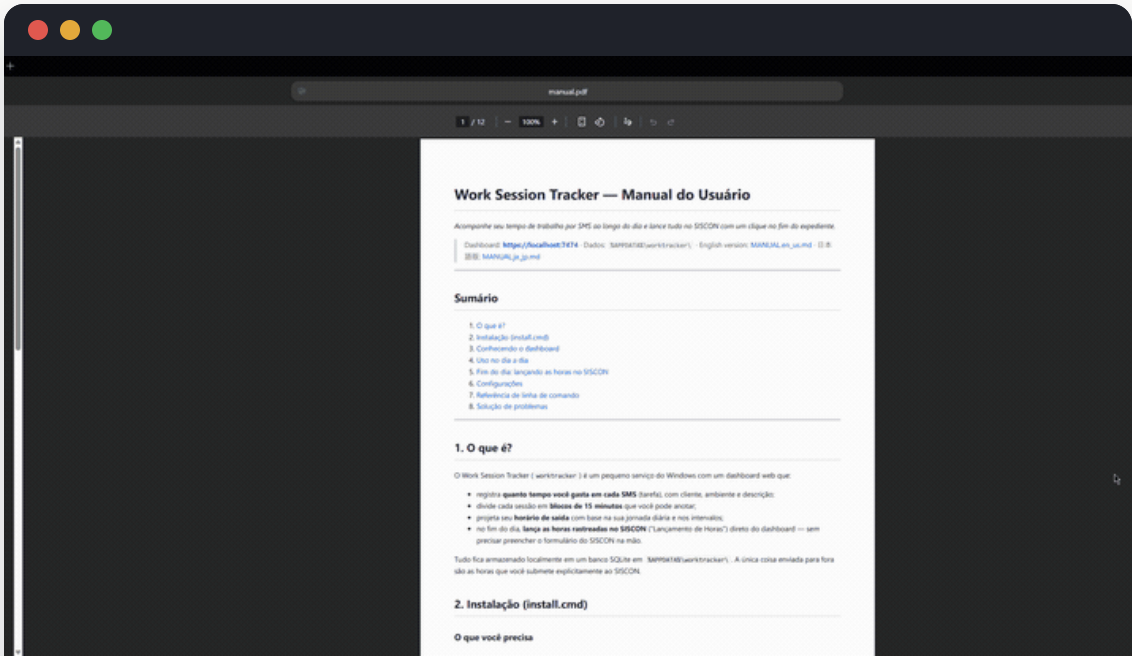
Unknown name? Pick from the whole disk

With Everything's es the picker searches every drive instantly; otherwise fd walks your configured roots.

Search everything a project holds, not just its code



g docs manual --all: every match inside a PDF is its own filterable row



Enter opens the document at the hit in its default viewer

ripgrep speed, fzf ergonomics

Every match is a row with a live preview. Tab marks several; Enter opens each in your editor at the line.

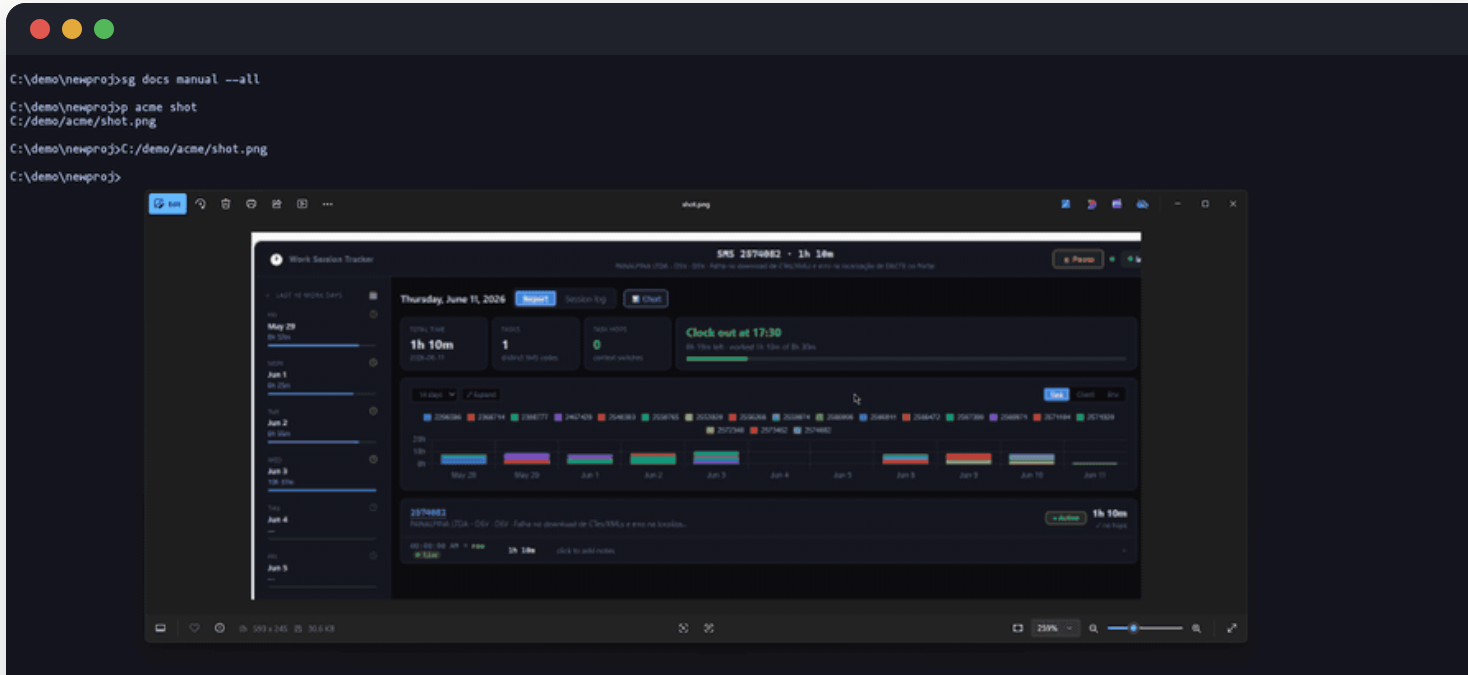
Inside documents with --all

ripgrep-all reaches into PDFs, Office files, archives and ebooks. Make it the default with one config line.

f and s share the picker

Find by name, or pick files and open each with its default app: "that invoice PDF" in two words.

Clipboard to file, from any prompt



p acme shot: the screenshot is saved as shot.png in acme, and its path is copied back

screenshot → **shot.png**

text → **shot.md**

Explorer files → copied in

clash → **shot-1.png**

Whatever is on the clipboard

An image becomes `.png`, text becomes `.md`, files copied in Explorer are copied in (folders recursively).

Never clobbers

`shot.png`, `shot-1.png`, `shot-2.png`. An explicit extension wins.

And the way back out

y acme invoice picks files and puts them on the clipboard as real files, ready to paste into email or chat.

Every project's commands, runnable from anywhere

```
actions.toml

# acme/.nix/actions.toml - committed with the project
[actions]
build = "zig build -Doptimize=ReleaseFast"
test  = "zig build test"

# Dev server with hot reload on :5173.
serve = "npm run dev -- --port {args}"

# Mirrors dist/ to the live host. Not reversible.
deploy = "npm run build && rsync -a dist/ host:/srv"
```

```
any directory

> x acme :test
> x acme :serve 8080
> x acme :build :test :deploy # stops at first failure
> x acme -o :serve            # its own window
> x :                        # every project's actions
```

Like package.json scripts, for any stack

Plain shell lines: pipes, `&&`, redirects. `[bash]` and `[pws]` sections for shell-specific ones.

Self-documenting

The comment above an action becomes its description in every listing, and is searchable: `nix --actions "not reversible"`.

Chains, arguments, windows

`:a :b :c` runs in order and stops at the first failure. `{args}` places arguments mid-command. `-o` gives a detached console.

The palette

`x :` gathers every project's actions into one picker. Tab-mark several and they start in parallel, each in its own window.

Three layers: shared by the team, private to you, machine-wide

	Project committed, travels with <code>git clone</code>	Central private to you, in <code>~/.nix</code>	Machine-wide every alias on this machine	Precedence
actions	<code>.nix/actions.toml</code>	<code>actions/acme.toml</code>	<code>actions/_default.toml</code>	<i>most specific wins</i>
environment	<code>.nix/env.toml</code>	<code>env/acme.toml</code>	-	<i>private layer wins</i>
sub-paths	<code>.nix/segments.toml</code>	<code>segments/acme.toml</code>	<code>segments.toml</code>	<i>nearest wins</i>
scripts	<code>.nix/scripts/</code>	<code>scripts/</code>	-	<i>project shadows central</i>

Teams share the defaults

The repo commits how to build, test and serve, plus dev defaults for its environment.

You keep your overrides

Your database, your staging box, your shortcuts: none of it ever shows up in `git status`.

Habits follow you everywhere

`_default.toml` defines `:st`, `:lg`, `:fresh` once, for every alias you will ever register.

direnv for Windows, with credentials kept out of files

two files, one environment

```
# acme/.nix/env.toml (committed defaults)
[env]
DATABASE_URL = "postgres://localhost/acme_dev"
API_BASE     = "https://staging.internal"
ACME_TOKEN   = "${secret:acme-api}"

# ~/.nix/env/acme.toml (private, wins per key)
[env]
DATABASE_URL = "postgres://box.local:5433/acme"
```

provenance

```
> nix acme --env
```

NAME	FROM	VALUE
ACME_TOKEN	project	\${secret:acme-api}
API_BASE	project	https://staging.internal
DATABASE_URL	central	postgres://box.local:5433/acme

`nix --secret set acme-api`
prompted, never echoed



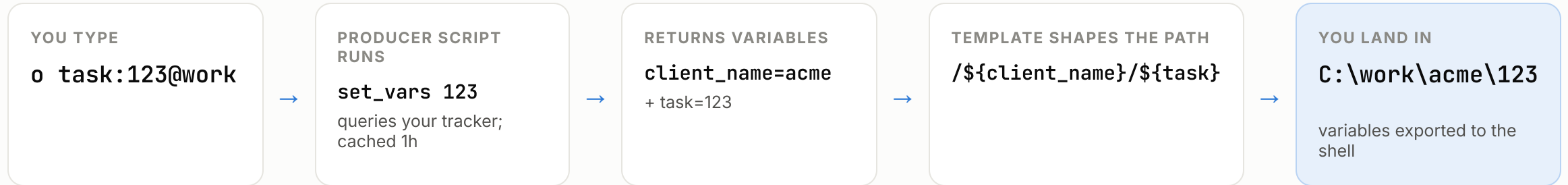
Windows Credential Manager
no secret ever sits in a file



resolved at spawn time
only in the child's environment

- ✓ Every `o`, `x` and `:action` gets the variables. Nothing to source.
- ✓ A missing secret aborts a run before it starts: no half-configured success.
- ✓ Elevated commands never receive secrets on a readable command line.

Paths that look things up, so you don't have to



Static sub-paths for every project

```
segments
> o docs@acme      # acme/documentation
> g src@api panic  # search one subtree
> p tasks:432@acme # file into tickets/432
```

A script can answer with a menu

```
o task@work
> PROJ-123 Fix login flow
   PROJ-140 Rate limiter
   PROJ-151 Export to CSV
```

Several answers open a picker; one navigates silently. Named **producers** let one lookup serve many projects with different folder layouts, sharing one cache.

Variables resolve from

1 · the inline value :123

2 · the script's output

3 · the environment

4 · [contexts.vars] defaults

Turn any project tool or action into a command on PATH

DECLARE, IN THE PROJECT

```
[bin]
hoot = "bin/hoot.exe"
ship = ":deploy"
```



REVIEW AND ALLOW

```
nix --sync-bin
per name, per version (content hash)
```



INSTALLED

```
~/.nix/bin/ship.exe
already on PATH; no PATH edits
```



ANYWHERE

```
ship --prod

= x acme :deploy -- --prod
```

```
_default.toml

# ~/.nix/actions/_default.toml - personal, no project
[actions]
gs  = "git status -sb"
sudo = "sudo {args}"
cc  = "cd | clip"

[bin]
gs = ":gs"
su = ":sudo"
cc = ":cc"
```

Files or actions

Export a built tool, a script, or a saved action. An exported action still runs in its project, with its environment, notifications and failure hold.

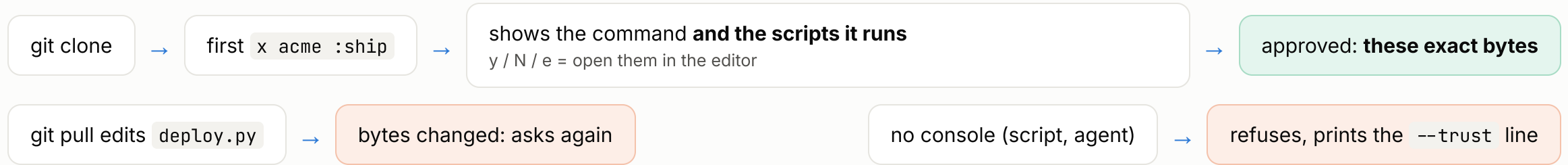
Replaces profile glue

Personal commands in `_default.toml` (`gs` , `su` , `cc`) work identically in `cmd` and `PowerShell`, declared in one greppable file.

Can't rot

Remove the line and the command is uninstalled on the next sync. Name collisions are refused; hand-edited copies are restored; `--doctor` reports drift.

Code that arrived with a clone never runs unread



```
the gate, as you see it

nix: acme's :ship wants to run:
  python tools/deploy.py --prod
  declared in C:\code\acme\nix\actions.toml
  runs      C:\code\acme\tools\deploy.py
Approve these files as they stand, and run? [y/N/e=open in editor]
```

Consent, not a name

Typing `:build` is not consent to whatever a clone says `:build` does. Actions, scripts, `env.toml` and lookup scripts are all gated.

Per action, per byte

Editing `:deploy` re-arms only `:deploy`. Re-approving supersedes, so reverting to old bytes still asks.

Elevation is always shown

UAC names the shell, not the command. nix shows the exact administrator line before every elevated run.

Standing trust for your repos

`--trust acme --always`, granted at a console, recorded in config, listed by `--doctor`.

Built to be composed, not just typed

Completion notifications

`[notify] on_finish` reports every action's outcome to any notifier. Failures always report; fast successes stay quiet.

Shortcuts that don't lie

Pin `x acme :build :test` to the taskbar. On failure the window holds with the error instead of vanishing.

Fully scriptable

Without a console every picker prints a table instead of blocking. `--list-names`, `--no-prompt`, `--json`.

Path dialects

`nix acme --as wsl` → `/mnt/c/code/acme`.
Also `gitbash`, `slash`, `uri`.

Prompt integration

`nix --which` and `$NIX_ALIAS` tell a prompt or status line which project you are in.

Time ledger

Every session and run is written to `~/.nix/time`: where the week went, by project, with nothing leaving the machine.

notify on failure

```
# ~/.nix/config.toml
[notify]
on_finish      = 'hoot send "{message}" --level {level}'
on_finish_min_ms = 2000
```

one line, every project

```
> nix --list-names | % { x $_ git pull --ff-only }
# every registered repo, updated in one line
```

Designed for developers and their coding agents

Coding agents learn the same command surface you use, so they say `x acme :test` instead of guessing build commands or quoting absolute paths.

```
wiring an agent

# ~/.claude/CLAUDE.md
@~/.nix/AGENTS.md

> nix --agent segments  # full spec + safety tier
> x --agent
> p shared@acme spec    # hand an agent a file
```

Generated, never stale

`~/.nix/AGENTS.md` is regenerated on every sync and shows this machine's own command names.

Specs on demand

`<cmd> --agent` renders the full contract and a safety tier for each command, from the same table that drives `--help`.

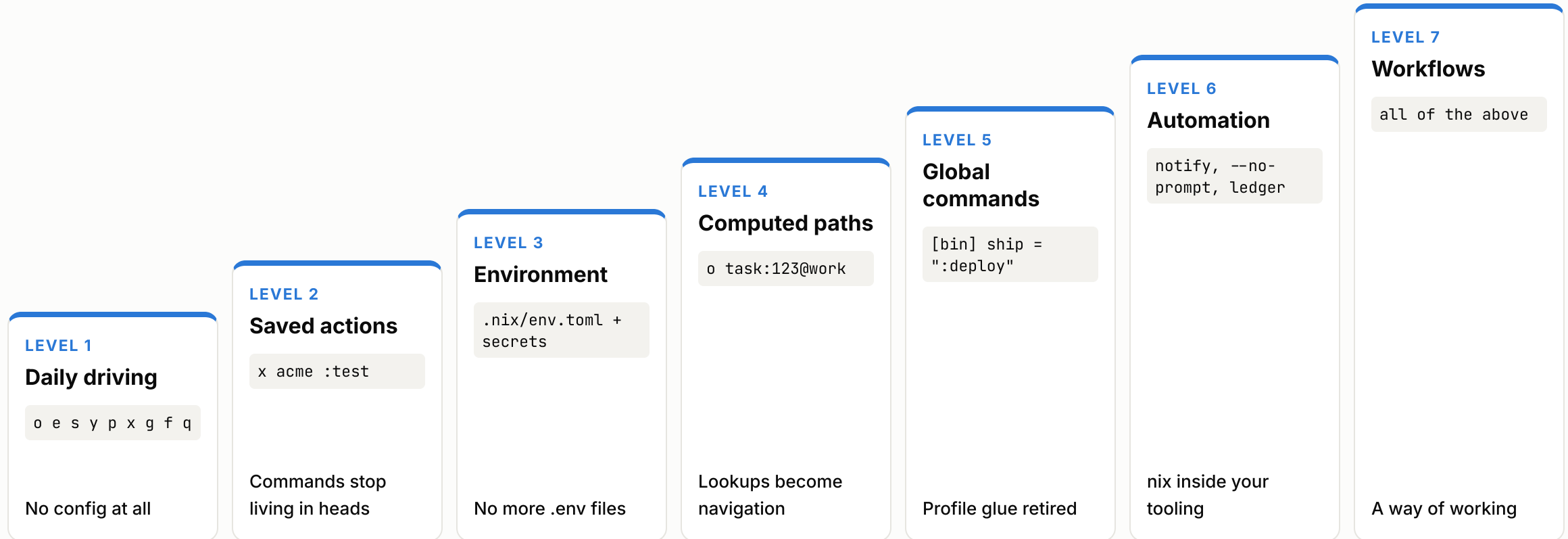
A consent boundary agents can't cross

An agent's shell has no console, so it cannot approve project code, including code it just wrote. The person approves; the agent runs.

A handoff folder per project

`shared@acme` is a built-in drop for notes and files between you and your agents.

Start with one alias. Grow into a platform.



Value from the first minute; depth when you want it. Every level is useful on its own, and each one is a file you can read.

What it looks like put together

Ticket-driven work

```
> o task@work  
> p task:123@work repro  
> g task:123@work "exception" -a  
> x task:123@work claude
```

Pick today's ticket from a menu, file evidence, search attachments, start an agent in the right folder.

Onboarding a cloned repo

```
> nix acme C:\code\acme  
> x acme :  
> nix --trust acme  
> nix acme --env
```

See what it can do, review everything once, run with private overrides.

Multi-service platform

```
> x plat :gen :build :e2e  
> x api@plat -o npm run dev  
> g web@plat "useAuth"
```

One alias, segments per service, one chain per story.

The morning start

```
> nix --list-names | % { x $_ :fresh }  
> x : (Tab: api, web, worker)  
> o api
```

What moved overnight everywhere; bring the stack up from one picker.

Seconds per task, compounded across an organization

ILLUSTRATIVE MODEL, NOT A MEASUREMENT

$$\text{hours / year} = \text{commands per day} \times \text{seconds saved} \times 230 \text{ working days} \div 3600$$

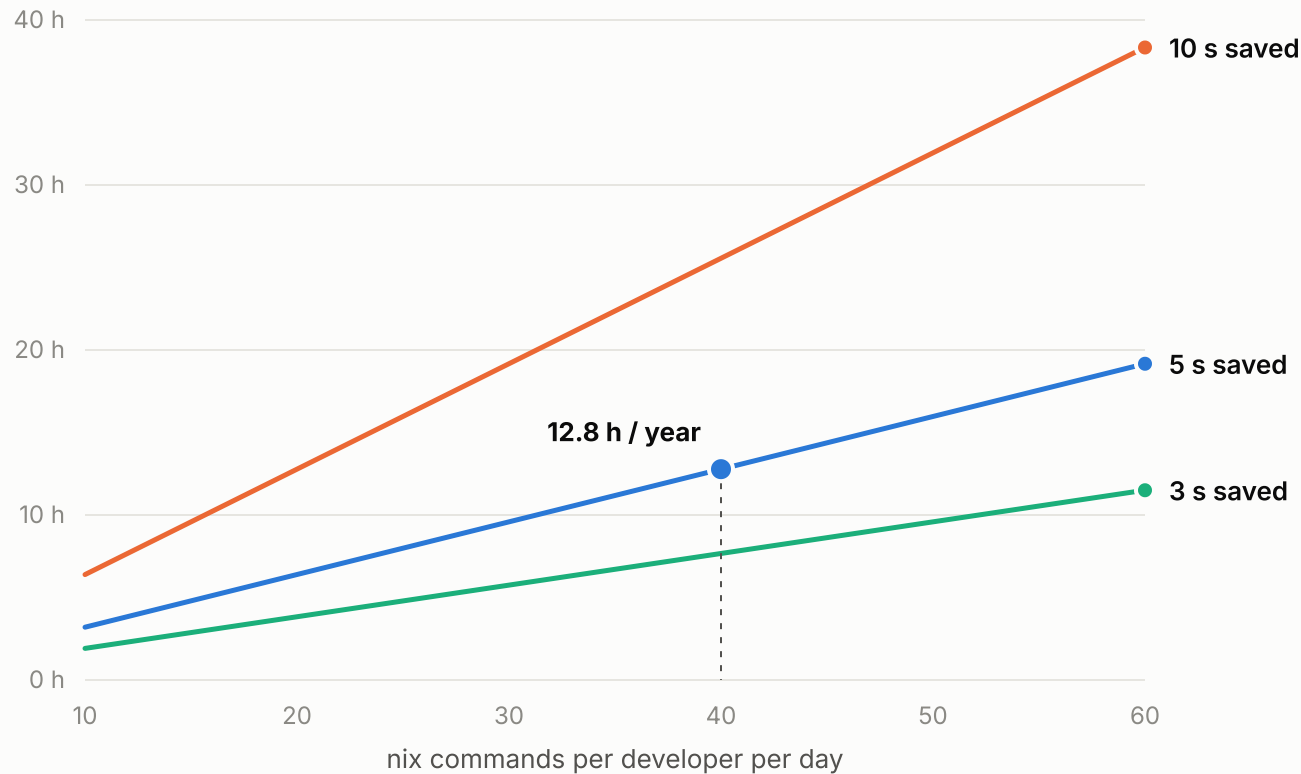
12.8 h

per developer per year at 40 commands a day, 5 seconds saved each

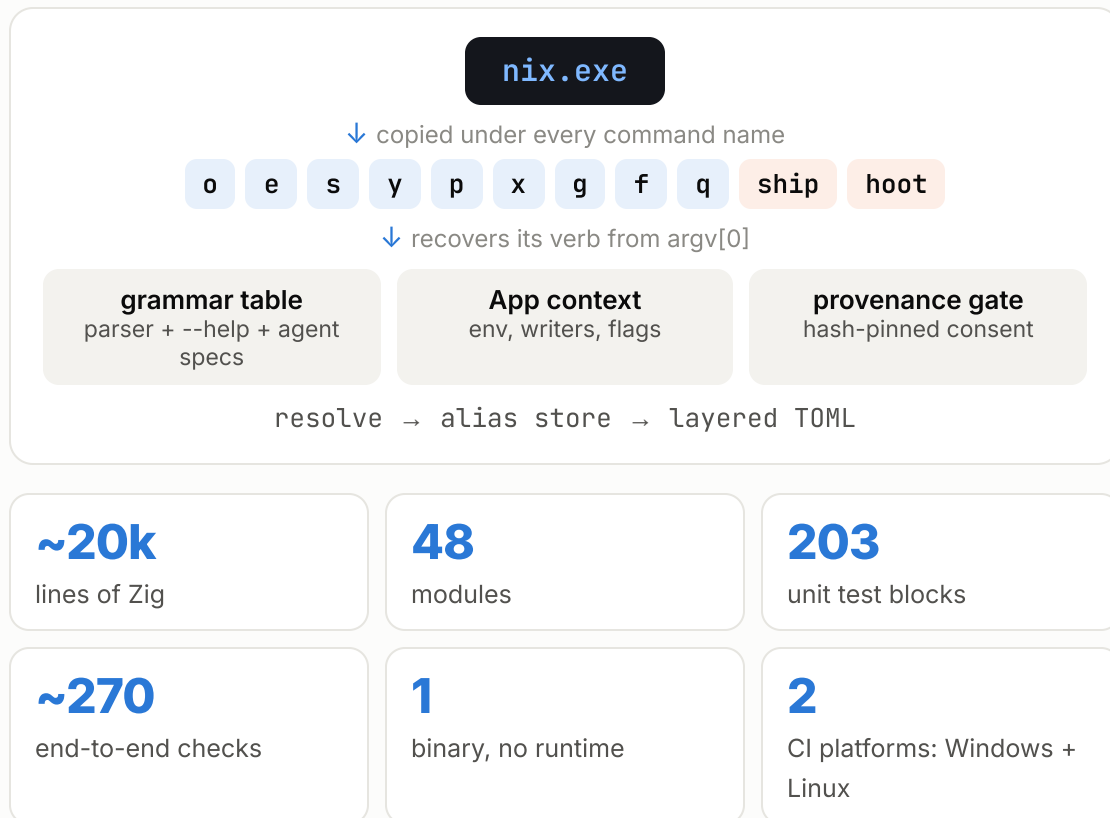
~12,800 h

per year across 1,000 developers, on the same inputs

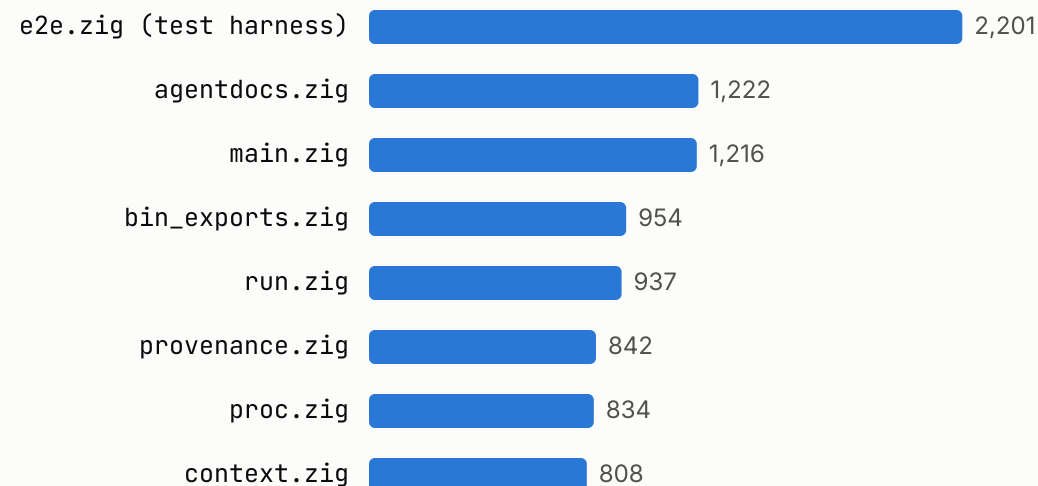
Replace the inputs with your own: count the `cd`, path copies and build commands in a week of shell history. The keystroke comparison on slide 4 is the measured part.



Small, auditable, and tested end to end



Where the code goes (largest modules, lines)



A single source-of-truth grammar table feeds the parser, `--help` and the agent specs, so documentation cannot drift from behavior; the dispatcher switches over it exhaustively, making an unwired command a compile error.

A low-risk path from one laptop to the whole organization

PHASE 1

Pilot

Install via Scoop (pulls fzf, ripgrep, fd, bat)

Register daily aliases

Personal `_default.toml` actions

`nix --doctor` in the onboarding checklist

PHASE 2

Team

Commit `.nix/actions.toml` and `env.toml` per repo

Secrets referenced as `${secret:NAME}`

Shared segments for docs, services, tickets

A `[notify]` hook into team chat

PHASE 3

Organization

Named producers for the ticket system

`AGENTS.md` wired into agent configs

Policy for standing trust on owned repos

Nightly or stable channel via Scoop

Get started in three commands.

```
> scoop bucket add sadirano https://github.com/sadirano/bucket  
> scoop install nix  
> o acme C:\code\acme
```

Guide

docs/GUIDE.md: every use, from daily commands to complete workflows

Recipes

COOKBOOK.md: copy-paste global commands

Reference

README.md: exact semantics and the reasoning behind them

Source

github.com/sadirano/nix · MIT licensed · built in public CI